Önálló laboratórium beszámoló

Távközlési és Médiainformatikai Tanszék

Készítette:	**Bacsó Janka Édua**
Neptun-kód:	**A6VVDJ**
Ágazat:	**Infokommunikáció**
E-mail cím:	**janka.bacso@edu.bme.hu**
Konzulens(ek):	**Papp Dávid**
E-mail címe(ik):	**pappd@tmit.bme.hu**

Téma címe: Kép generálás létrehozott adathalmaz alkalmazása CNN architektúra betanítására

Feladat

A félév folyamán megismerkedünk a kép generálással és annak egy felhasználásával. Egy CNN architektúrát kép generálás segítségével létrehozott adathalmazon tanítunk be. Vizsgáljuk ennek eredményeit és összehasonlítjuk a CIFAR10 adatbázison betanult modell eredményeivel. Saját körülmények között megnézzük mennyire lehet ez hasznos alkalmazási terület.

2022/2023. 2. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1 Bevezető

Manapság a kép generáló mesterséges intelligenciák egyre felkapott témának számítanak. Önálló laboratóriumom során ki szerettem volna próbálni ennek a szolgáltatásnak egy lehetséges alkalmazását.

A generált képek felhasználása - mint betanító adathalmaz képfelismerő modellben - lett a felhasználási mód, amit megvalósítottam. Ez azért lehet hasznos, mert ha esetleg szeretnék betanítani egy modellt, de nincs elég kép a birtokunkban egy adott témában, akkor egy generált adathalmaz segítségével kiegészíthetjük. Esetleg ha saját modellt tervezünk építeni specifikusabb témában, ennek segítségével kreálhatunk betanító képeket.

1.2 Elméleti összefoglaló

1.2.1. Kép generálás

Az AI kép generálás egy felkapott téma manapság. Ezek közül is a DALL-E [1]talán a leghíresebb, amit az OpenAI[2] fejlesztett. A kép generátor bemenetként megkap egy leírást (promt), ami alapján generál képeket számunkra.

1.2.1.1 Hogyan működik?

Annak érdekében, hogy egy ilyen AI betanuljon, rengeteg képre van szüksége. Ezeken a képeken mindenféle dolog szerepelhet, arra kell fokuszálniuk, hogy minél változatosabbak legyenek azzal kapcsolatban, amilyen témában szeretne generálni később képeket.

A mesterséges intelligencia a tanulási folyamathoz gépi tanulást használ. A modell megtanulja a képek eleminek bizonyos jellemzőit, adatait. Ezeket használja fel később a generálás során.

Ha rendesen betanult, onnantól kezdve képes bemenetről bekapott információk alapján visszaadni számunkra képeket. A generálás során a bementként kapott információkat használja fel, hogy a megtanult jellemzők alapján alakítsa és kombinálja azokat.

1.2.1.2 Milyen felhasználási módjai lehetnek?

A kép generálás több szempontból is hasznos lehet számunkra. Egyese esetekben például nagyobb cégek időt és pénzt tudnak spórolni azzal, ha generálnak maguknak képeket szükség esetén. Például egy divat cégnél egyszerűbb legenerálni pár design ötletet ahelyett, hogy felvegyenek még tervezőket. Illetve egy hirdetés megtervezése is egyszerűbb lehet így. A film- és a játékiparban is hasznos lehet például karakter design készítésben. Emellett sokkal nagyobb mennyiségben lehet képet generálni adott idő alatt, mint megtervezni.

1.2.2. Kép felismerés

Sokféle modell létezik, amit kép felismerésre lehet használni, és egy modellen belül sok mindent lehet beállítani. A cél, hogy minél jobb arányban ismerjen fel az AI képeket. Ehhez hatalmas mennyiségű tanító adatra van szükség.

1.2.2.1 Lépései

Először is kell egy adathalmaz, amin be fog tanulni a modellünk. Ez általánosságban egy -

[1] DALL-E gépi tanuló mesterséges intelligenciák, amiket az OpenAI készített
[2] OpenAI egy mesterséges intelligencia kutatólaboratórium

már létező - képgyűjtemény szokott lenni. Esetünkben viszont ezt helyettesítjük generált képekkel. Ha létező halmazt használnánk, rengeteg kép lenne birtokunkban, most viszont limitált forrásaink vannak, így kisebb mennyiségű képet fogunk felhasználni.

Következő lépés csak a mostani feladatra vonatkozóan történik, a képeinket adatokká kell alakítanunk. Azért van szükségünk erre, mert ez a képeket összekapcsolja a kategóriáikkal, emellett olyan alakba teszi őket, ami később beadható a modellünknek.

Adat augmentációra is szükség van egy tanulás során, hogy semmiképp ne csak egy fél típusú képre tanuljon be a modell. Alapesetben ezt a modell tanító adathalmazán és validációs adathalmazán megtesszük, mielőtt elindítjuk a tanítást. A mi esetünkben viszont ez a modellbe helyezve történik meg.

Ezután megépítünk egy modellt, rétegeket egymásután hozzáadva. A különböző kép felismerők, ebben el tudnak térni egymástól. Számít, hogy hogyan épülnek fel a rétegek. Most AlexNet-et fogjuk használni.

Az AlexNet 8 rétegből áll : 5db convolutional rétegből és 3db fully connected rétegből. Emellett 3 maxpooling is van még az architektúrában. A túltanulás kezeléséhez használ még Dropout függvényt. Dropout egy olyan metódust hív meg, aminek során minden neuron kap egy valószínűségi szorzót, hogy kiesik-e az architektúrából a képfeldolgozás során, vagy nem. Kiesés azt jelenti, hogy nem vesz részt az adat osztályozás/elemzés során. Ez minden adatnál változik, így a különböző adatok, az architektúra különböző verziói alapján lesznek osztályozva. Így sokkal nehezebb egy bizonyos értékre beállnia a rendszernek. A különböző verziók súlyai viszont ugyanzok maradnak.

Ha megvan a modellünk, megépítjük. Több argumentuma is van ennek a függvénynek, például itt beállíthatjuk az optimalizálásának logikáját, vagy a veszteség függvényt, amit használjon a veszteség kiszámolására.

Ha a modellünk kész, elkezdhetjük a tanítást, és ha az lefutott, lehet tesztelni.

1.3 A munka állapota, készültségi foka a félév elején

Előző félévben elvégzet témalabor során, megírtam egy CNN architektúrát (AlexNet) és megismerkedtem a kép felismeréssel. A jelenlegi feladat során felhasználtam az általam írt AlexNet-et, és azt egészítettem ki, hogy az általam generált képekkel dolgozzon.

2. Az elvégzett munka és eredmények ismertetése

1.1 Képek generálása és a modell betanítása

A feladat során a OpenAI API3-jának használata mellett döntöttem. Átlátható és könnyen értelmezhető dokumentációval rendelkezik.

```
#Generating images with openai and save them
for p,p2 in zip(prompts,prompts2):
  time.sleep(15)
  newPath = os.path.join(myPath, p)
  os.makedirs(newPath, exist_ok=True)
  response = openai.Image.create(
      prompt= p2,
      n=10,
      response_format='url',
      size="256x256"
      )
  if "data" in response:
    for key, obj in enumerate(response["data"]):
      filename ='my_image_'+str(key)+".png"
      fullfilename = os.path.join(newPath, filename)
      urllib.request.urlretrieve(obj['url'], fullfilename)
    print('Images have been downloaded and saved locally')
  else:
    print("Failed to generate image")
```

1. ábra Kép generálás algoritmusa

A kódban egy ***for*** ciklusba helyeztem bele a generálást, hogy minden kategórián végighaladva generáljon tíz képet. Tíz kategóriánk van, így ez csak 100 képet eredményezett volna. Viszont ezt a folyamatot megcsináltam még ötször, hogy 500 képpel rendelkezzünk. Sajnos, ahogy a kódban is látható [1], az n (darab szám) értéke tíznél nagyobbra nem állítható. Végül 510 képet használtam fel.

A képek ki lettek mentve, hogy ne vesszenek el, és többször felhasználhatóak legyenek.

Miután képek generálásával megvagyunk, adathalmazt kell belőlük csinálnunk, amin egy modell be tud tanulni. A ***tf.keras.utils.image_dataset_from_directory***4 függvényt használtam a képek átalakításához.

```
pathfordata = "/content/drive/MyDrive/onlab/pics"

x_train =tf.keras.utils.image_dataset_from_directory(
    pathfordata,
    color_mode="rgb",
    batch_size=32,
    image_size=(32, 32),
    shuffle= True,
    seed = 13,
    validation_split= 0.2,
    subset = "training"
)

x_val =tf.keras.utils.image_dataset_from_directory(
    pathfordata,
    color_mode="rgb",
    batch_size=32,
    image_size=(32, 32),
    shuffle= True,
    seed = 13,
    validation_split= 0.2,
    subset = "validation"
)

y_train = x_train.class_names
y_val = x_val.class_names

print(x_train)
print(x_train.class_names)
```

2. ábra Képekből adathalmaz (Dataset)

Az első argumentum, amit megkap a függvény (*pathfordata*), az az útvonal a mappához, ami tárolja őket. Ezen a mappán belül kategóriákra bontva külön-külön mappában vannak a képek. A függvény ezáltal tudja kiszedni a kategóriák neveit. Az almappa neve megegyezik az kategória/osztály nevével (*class_name*). Ezeket majd külön az „**y_**” előtaggal ellátott változókban tároljuk le.

Az **image_size** értékét 32x32-ére állítjuk. Bár 256x256-os méretű képeket tudtunk generálni, a CIFAR10[5] adathalmaz 32x32-es képekkel rendelkezik. Mivel tesztelni szeretnénk vele, egyszerűbbnek véltem a generált képeket csökkenti méretben, mint fordítva. Azért kell egy méretűeknek lenniük, mert a modell csak egy bemeneti mérettel tud működni.

A **shuffle** argumentum **True** értékre való állítása arra szolgál, hogy az adataink ne sorrendbe legyenek állítva kategóriánkként az adathalmazban.

[3] API - Application Programming Interface
[4] tf.keras – implementációja a Keras API-nak, magasszintű Tensorflow
[5]CIFAR10 - A CIFAR-10 adatkészlet olyan képek gyűjteménye, amelyeket általában gépi tanulási és számítógépes látási algoritmusok betanítására használnak.Az adatkészlet 60 000 32x32-es színes képet tartalmaz 10 különböző osztályban.

A **seed** értéke egy bármilyen szám lehet. Jelentősége, hogy amikor tanító és validációs adathalmazt készítünk nekik, ugyanaz a szám legyen az argumentumban megadva. Ez azért kell, hogy a két halmaz ne válassza ugyanazokat a képket.

A **validation_slipt** adja meg hány kép tartozik a validációt adathalmazhoz. Az esetünkben 0.2 adtunk meg, ami azt jelenti, hogy a képek 20%-a lesz benne (103 kép). A többi kép a tanító halmazba megy.

A **subset** adja meg a függvénynek melyik adathalmazról beszélünk. A tanító halmaz esetében „*training*”, a validációs adathalmaz esetében pedig „*validation*” értéket kap.

```
#1st Convolutional Layer
AlexNet.add(data_augmentation)
AlexNet.add(Rescaling(1./255))
AlexNet.add(Conv2D(filters=96, kernel_size=(11,11), strides=(4,4), padding='same'))
```

3. ábra A modell első 3 sora

A modellemet is megváltoztattam annyival, hogy az első két sorába új rétegeket tettem. Az első az adat augmentációt tartalmazza. A második sor pedig a **Rescaling** függvényt. Ennek segítségével a 0-tól 255-ig terjedő RGB értékeket 0-tól 1-ig terjedő értékekké változtatja.

Az augmentációt saját függvénnyel hajtjuk végre:

```
data_augmentation = keras.Sequential(
  [
    keras.layers.RandomFlip("horizontal",
                            input_shape=(32, 32, 3)),
    keras.layers.RandomRotation(0.1),
    keras.layers.RandomZoom(0.1),
  ]
)
```

4. ábra adat augmentáció

A **keras** tartalmaz előre megírt függvényeket, ezeket használjuk fel. Erre a lépésre azért van szükség, hogy ne csak egyféleképpen kinéző képre tanuljon be a modell. (Ne csak függőlegesen elhelyezett kutyát ismerjen fel). A bemenet méretét is itt adjuk meg: **(32,32,3)**

Ezek mellet a modell veszteség(*loss*) függvényét is ki kellett cserélnem, hogy az általam beadott adatokkal is működjön (*SparseCategoricalCrossentropy*). A veszteség függvény megadja a mértéket, amivel csökkenteni kell a számítási értékeit, hogy jobb eredményt érhessen el.

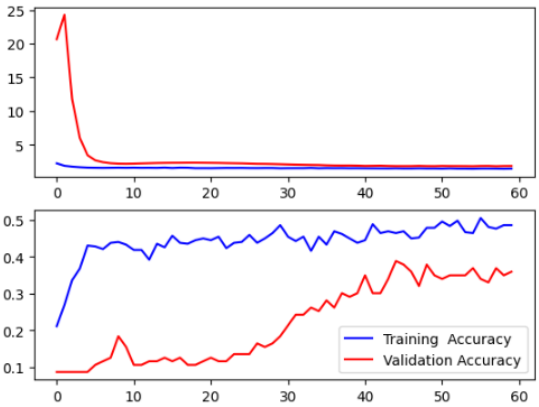
A modellünk és az adathalmazunk ezen függvények és átalakítások segítségével működni fog

1.2 Eredmények

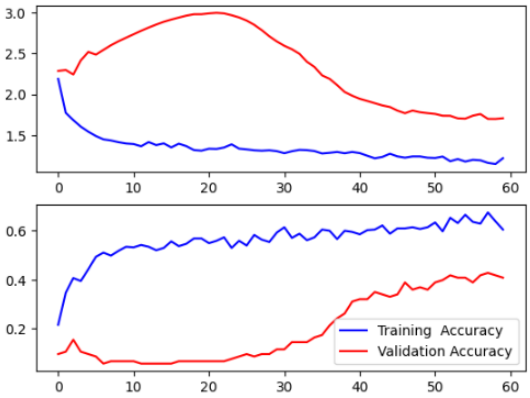
A tesztelések során kétféle optimalizálásra is megnéztem a modellt, ezek pedig **adam** és **Nadam** voltak. Az optimalizáció a bemeneti súlyok beállításáért felel. Összehasonlítja a veszteség függvényt és a várt értéket, és az eredmény alapján változtatja a súlyokat. Összehasonlítottam mennyire teljesítenek jól egymáshoz képest.

Az alább látható ábrákon [5],[6] a felső kvadránsban a veszteségek egymáshoz viszonyulása. **Adam** során elég hamar beáll egy alacsony értékre, ami azt jelenti hogy minimális változtatás történik. Viszont figyeljük meg az oldalsó értékek 0-tól 25-ig terjedő skálán vannak. **Nadam**-nál viszont 0-tól 3-ig terjedő a skála. Bár van benne egy nagy ugrás, viszont számérték szempontjából maximális 1.5 a különbség. Jóval kisebb, mint **adam**-nál**.**

Az alsó kvadráns a pontosságot(*accuracy*) reprezentálja. A két optimalizáció egész hasonló eredmény mutat, de figyeljük meg, hogy a **Nadam** 0.6-os értéket is eléri a tanító értékkel, 0.4-et pedig a validációssal.



6. ábra Adam optimalizáció eredménye



5. ábra Nadam optimalizáció eredménye

Ezek természetesen nem a legszebb számok, és már itt látható, hogy nem a legjobb kép felismerő algoritmus a miénk. Viszont tartsuk szem előtt, hogy nagyon apró adathalmazon tanult, ahhoz képest, amin tanítani szokás egy modellt. Az itt látható ábrák javuló tendenciát mutatnak feltételezhető, hogy nagyobb adathalmazon jobban tudnának teljesíteni.

1.2.1 Tesztek generált képekkel

A generált képek tesztjei kicsit egyszerűbbek, mint a CIFAR10 tesztjei lesznek, ugyanis sokkal kisebb adathalmazzal dolgoztam. Mind a két optimalizálás esetében, ugyanazokat a képeket adtam be a modellnek.[6]

Egy érdekes jelenséget meg lehet figyelni a képeken, ami talán hátrányt jelenthet. Ha képeket generálunk megeshet, hogy a betanítás során olyan képeket is kapunk, amin nem csak a célobjektum szerepel. Így lehet bezavarhat a modellünk tanulásába. (pl.: **ship**-dokumentum végén.) Tesztelésnél ez a probléma már kevésbé áll fenn, ugyanis itt csak az zavarhat be, ha több kategóriába tartozó objektum szerepel a képen.

Az alábbi táblázatban látszanak a tesztelések eredményei. Ezek az alapján születtek, hogy osztályozás során a képek a tíz kategória közül, melyikben érték el a legnagyobb százalékot. Annak a kategóriának nevét és a százalékát látjuk a táblázatban.

Teszt eredmények					
Adam	eredmény	biztosság	**Nadam**	eredmény	biztosság
Airplane	airplane	16.25%	Airplane	airplane	17.41%
Bird	bird	15.6%	Bird	bird	16.33%
Car	car	15.02%	Car	car	14.32%
Cat	cat	11.45%	Cat	frog	11.2%
Deer	bird	12%	Deer	cat	13.38%
Dog	horse	13.01%	Dog	horse	11.93%
Frog	cat	11.38%	Frog	cat	13.18%
Horse	car	12.51%	Horse	horse	12%
Ship	dog	11.13%	Ship	dog	11.77%
Truck	cat	14.67%	Truck	cat	19.54%

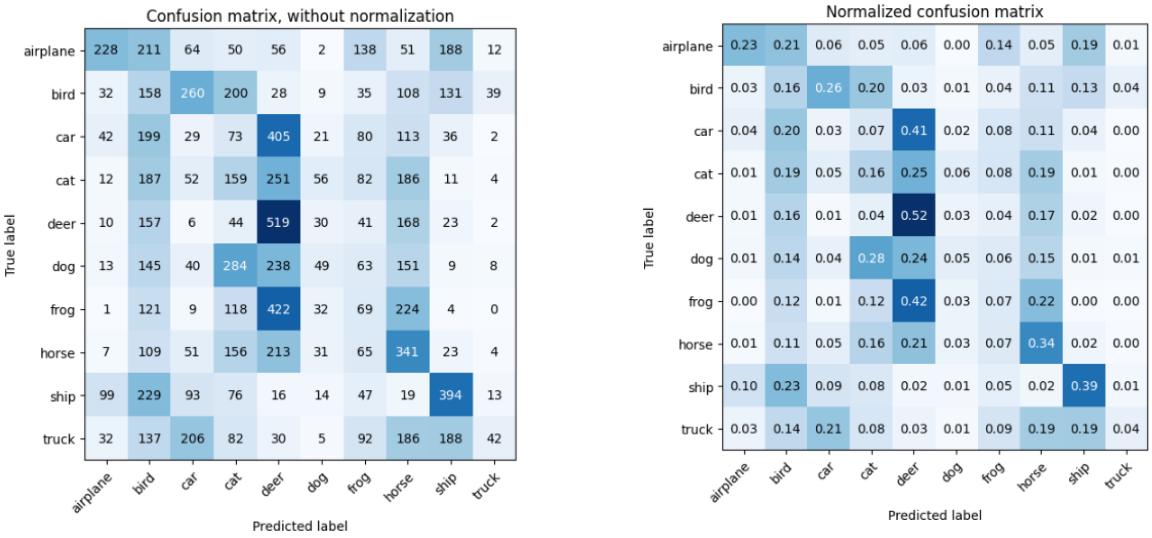
Az eredményeken is látható a két optimalizáló különbségének eredménye. Százalékokat tekintve a **Nadam**-ot használó modell minimálisan, de biztosabb az eredményekben, mint a **adam**.

[6] A tesztek eredményeihez tartozó képeket a dokumentum végére beillesztve megtalálják.

1.2.2 Tesztek CIFAR10 képekkel

A generált képek után jobb képet kaphatunk modellünk helyességéről, ha egy nagyobb adathalmazon vizsgáljuk. Erre tökéletes a CIFAR10. Itt is megvizsgáljuk adam és Nadam optimalizálásra is. Viszont itt mivel több adatra nézzük, tudunk létrehozni konfúziós mátrixot az eredményekből, és vizsgálhatjuk az egész modell pontosságát.

Adam eredmények



7. ábra Adam modell: konfúziós mátrixok

A feljebb lévő képből [7] kiolvashatjuk, mely kategóriákat képes a legnagyobb sikerrel felismerni. A mátrix lényege, hogy számolja, egy kategóriát hányszor talál el a modell, de azt is számontartja, hányszor nem. Emellett azt is megmutatja, mi másnak lett titulálva a modell által a kép. Például: a *frog* 422 alkalommal *deer* kategóriába lett sorolva.

Ezek alapján azt lehet leolvasni az ábráról [7], hogy a *deer* kategóriában találta el a legtöbb elemet, vagyis arra tanult be a legjobban. A *ship* és a *horse* kategóriában is már kezd egy jó átlagot közelíteni. Míg a *frog* és *car* képekre sokszor hiszi, hogy *deer.*

A modellünk pontossága: 0.1988. Nem túl magas szám, de a kicsi adathalmazon való tanításnál jóval magasabb értéket nem is lehetne elvárni.

Nadam eredmények

8. ábra Nadam modell: konfúziós mátrixok

Ezen az ábrán [8] jól megfigyelhetjük a különbséget az előzőhöz [7] képest. Itt a *ship*

kategória a legerősebb, de nem annyira mint az *deer* volt [7]. Viszont a *deer* kategória itt is erősebben jelenik meg.

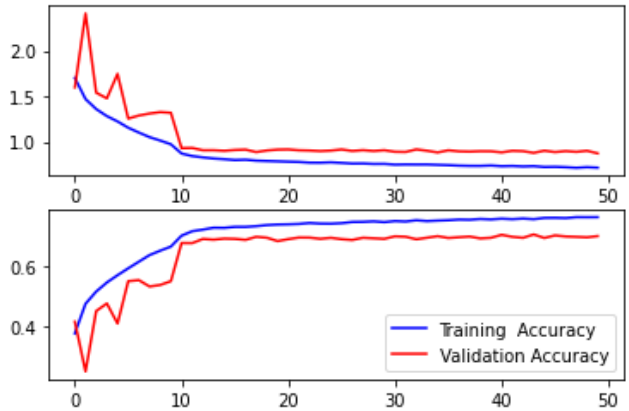
A modellünk pontossága: 0.2123. Jobb, mint amit az **adam**-mal értünk el.

1.2.2.1 Eredmények összefoglalása

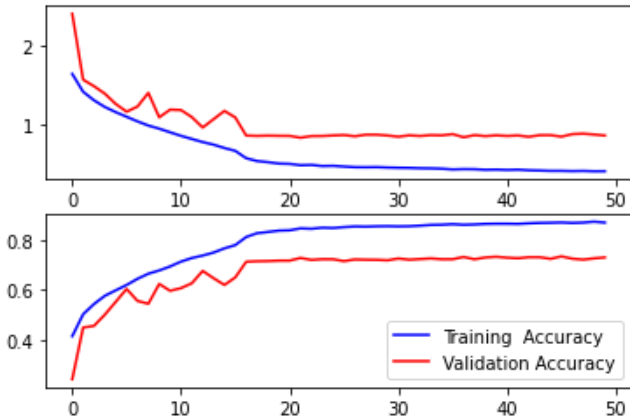
A konfúziós mátrixokból [7],[8] láthatjuk, hogy elég vegyesek az eredmények. Pár kiemelkedő kategória van, ahol sikeres kép felismerés tud történni, de azok sem a legbiztosabbak. Mind a két esetben látványosan sokszor a bird kategóriának tulajdonít képeket, mikor oda nem valóak. Itt megkérdőjelezhető, hogy esetleg a betanulás folyamán nem kapott ideális példákat.

1.2.3 Összehasonlítás CIFAR10-en betanult modellel

Az eddig eredményeket csak egymáshoz tudtuk nézni. Viszont biztosabb alapot nyújthat számunkra, ha egy jól működő modellel is össze tudjuk hasonlítani. Tavaly a feladatom egy CNN architektúra (AlexNet) írás és betanítása volt. Ugyanúgy teszteltem azt is különböző optimalizációs értékekre (**adam**, **Nadam**).



9. ábra adam optimalizáló eredmény

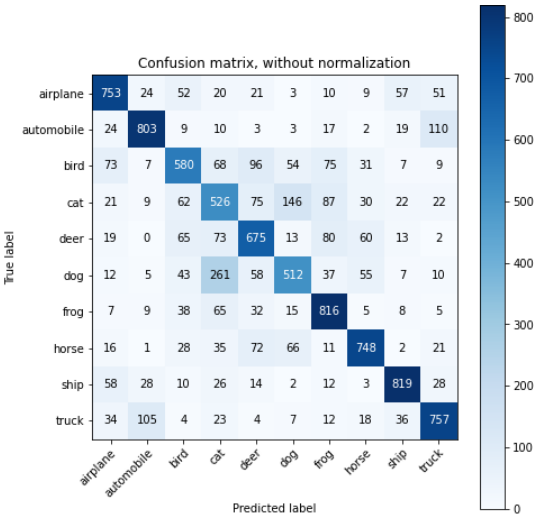


10. ábra Nadam optimalizáló eredmény

Figyeljük meg [9],[10] a **Nadam** esetében a pontosság(accurcy) értéke megint magasabbra ment mint az adam-é. Viszont a mi modellünkhöz képeset [5],[6] sokkal magasabbra ment. Nadam-nál a tanító 0.6-ra ment, míg itt 0.8-ra, a validációs 0.4-re ment, itt pedig alulról súrolja a 0.8-as értéket.

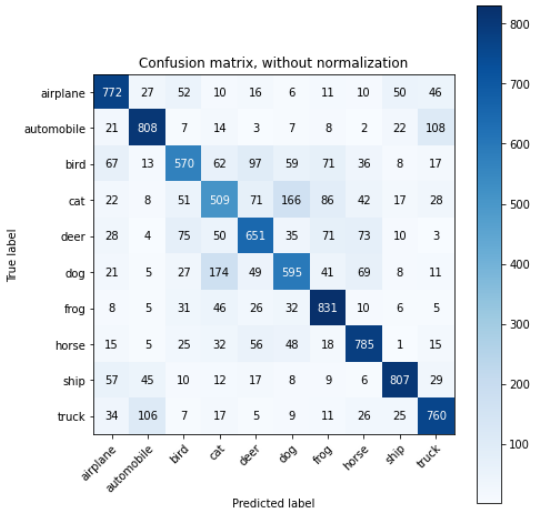
Ebből viszont csak annyi vonható le, hogy több adattal jobban tanul.

adam



11. ábra adam konfúziós mátrix CIFAR10 modell
pontosság: 0.6989

Nadam



12. ábra Nadam konfúziós mátrix CIFAR10 modell
pontosság: 0.7336

Itt láhatjuk, hogy egy sikeresen betanult modell esetében elég erősen az a vonal látható, ahol eltalálja a kategóriákat. A generált képes modell [8] ábráján már elkezd látszódni egy vonal, bár az látszódik, hogy vannak kategóriák, amik sikertelenek (*car, dog, truck*). Észrevehető az is, hogy sokkal kevesebb hibát ejt (kategorizál másnak egy képet) a CIFAR-on tanult.

2.2 Összefoglalás

Az elmúlt pár évben nagyon felkapott téma lett az AI és a kép generálás (DALL_E), engem pedig az érdekelt, lehet-e hasznos számomra azon kívül, hogy szép képeket generál. A félév folyamán az a kérdés foglalkoztatott, mennyire tud jól betanulni egy kép felismerő modell, generált képekkel. Ez hasznos lehet számunkra, ha esetleg nincs elég nagy adathalmazunk egy kategóriában, vagy ha egy specifikusabb képfelismerőt szeretnénk megalkotni.

Az előző félévben megismerkedtem a képfelismerés rejtelmeivel, és egy AlexNet modellt is írtam beadásként, így ezt fel tudtam használni az ötletemhez. Magát a modellt kiegészítettem és átírtam, hogy saját képekkel tudjak dolgozni rajta. Emellett az OpenAI API segítségével generáltam egy kisebb adathalmazt. Ezt alakítottam át a kódban úgy, hogy a modell be tudjon tanulni rajta.

Többféle tesztet is futtattam: kipróbáltam generált képeket milyen jól osztályoz, CIFAR10-es képeket mennyire jól ismeri fel, összehasonlítottam az előző feléves munkám eredményével is.

A kép felismerés nem volt teljesen sikertelen, viszont vannak kategóriák, amik látványosan gondot okoztak (pl.: *car, dog, truck*). Voltak sikeres eredmények is. Ez viszont változó volt attól függően, milyen optimalizálást használtam. Ha **adam**-ot nézzük, a *deer* kategória volt számára a legfelismerhetőbb. Ha **Nadam**-ot nézzük, a *ship* kategóriára tanult be jól.

Feltételezhető, hogy a betanulás folyamatát az akadályozta, hogy egyes generált képek nem elég konkrétak, túl keveset mutatnak, vagy túl sok mindent tartalmaznak (nem csak amit fel szeretnének ismerni).

Ez kiküszöbölhető lenne azzal, ha átnéznénk a képeket, de ezzel pont ellentmondanának az egész feladatnak. Az lenne a lényege a kép generálás alkalmazásának, hogy megkönnyítsük az adathalmaz létrehozását. Ha át kell néznünk őket, a munka mennyisége nem csökken, sőt ha nagy mennyiségben generálunk képeket, meg sem éri.

Természetesen azt is meg kell említeni, hogy nem egy nagy adathalmazon tanult be. Ez is beárnyékolhatta az eredményt. Emellett a felbontást is lejjebb kellett vennem, ami szintén hátrány jelentet. Viszont az nagyobb erőforrást kívánt volna részemről, ami nem áll rendelkezésemre.

A kép generálás napról napra jobban fejlődik. Már most is csodálatos képeket tudnak előállítani, viszont még nem tökéletes. Észrevehető, hogy egyes kategóriákban jobban teljesít, míg vannak olyanok amik még megakasztják (pl.: kezek). Emellett nehézséget okoz az is, hogy pontosan az kapjuk, mint amit szeretnénk.

3. Irodalom és csatlakozó dokumentumok jegyzéke

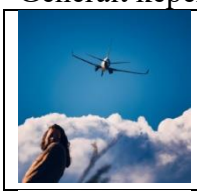
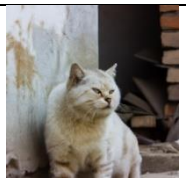
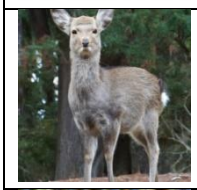
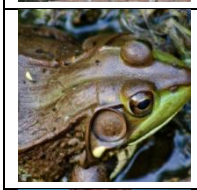
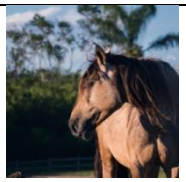
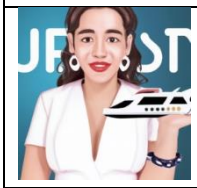
A tanulmányozott irodalom jegyzéke:

1	"Dall·E: Creating Images from Text." *DALL·E: Creating Images from Text*, https://openai.com/research/dall-e , Accessed 28 May 2023.
2	McFarland, A. (2023). Beginner's Guide to AI Image Generators. *Unite.AI*. https://www.unite.ai/beginners-guide-to-ai-image-generators/
3	Load and preprocess images. (n.d.). TensorFlow. https://www.tensorflow.org/tutorials/load_data/images
4	Image classification. (n.d.). TensorFlow. https://www.tensorflow.org/tutorials/images/classification

Csatlakozó egyéb elkészült dokumentációk / fájlok / stb. jegyzéke:

A féléves munka során keletkezett és a feladathoz felhasznált kód az alábbi linken érhető el:
https://colab.research.google.com/drive/1rSja-YW__OHxYNqAlg4l19v-gqkArqvR?usp=sharing

Generált képek, amiken a teszteket futtattam:

 airplane	bird
car	 cat
 deer	dog
 frog	 horse
 ship	truck